

Information and Network Security

Study Guide

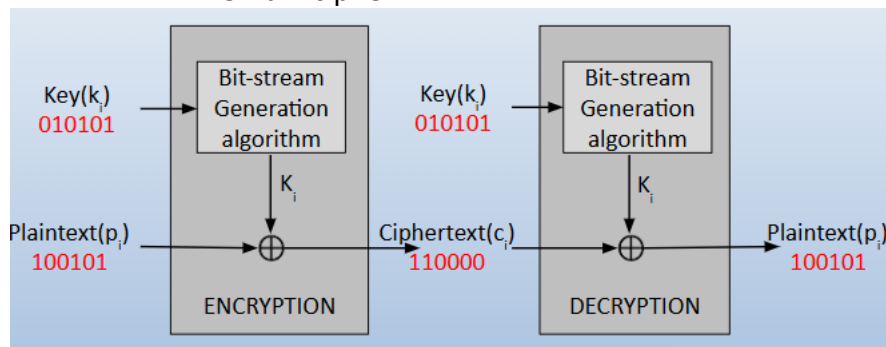
Praptiba Solanki
CSE, PIT
Parul University

Block Ciphers and the Data Encryption Standard:	3
3.1 Stream ciphers and block ciphers.....	3

3.1 Stream ciphers and block ciphers

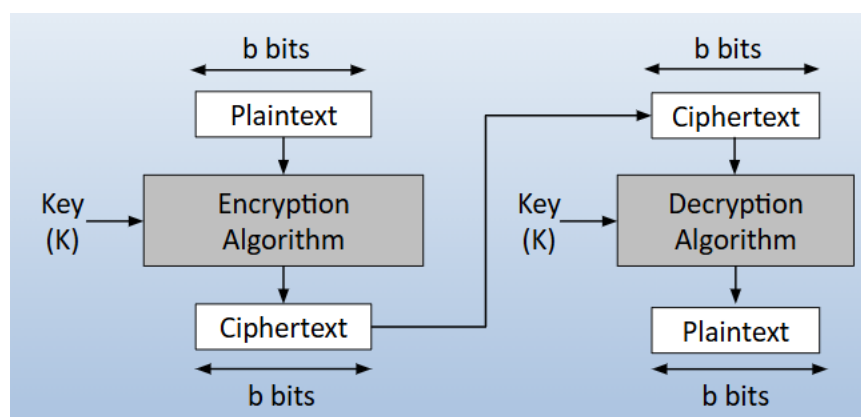
Stream Cipher:

- A stream cipher is one that encrypts a digital data stream one bit or one byte at a time.
- **Examples:**
 - Autokeyed Vigenère cipher
 - A5/1
 - RC4
 - Vernam cipher.



Block Cipher:

- A block cipher is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length.
- Typically, a block size of 64 or 128 bits is used.
- **Examples:**
 - Feistel cipher
 - DES
 - Triple DES
 - AES



References:

1. Cryptography and Network Security (Textbook) William Stallings; Pearson Education
2. Cryptography & Network Security Behrouz A. Forouzan; Tata McGraw-Hill
3. Information Security Principles and Practice Deven Shah, Wiley-India
4. Information Security Principles and Practice Mark Stamp; Wiley India edition
5. Information systems security Nina Godbole; Wiley Publications,200

3.2 Block cipher principles

Key Principles

Number of Rounds

The security of a block cipher is significantly influenced by the number of rounds of encryption it performs.

Each round applies a series of transformations to the data, making it increasingly difficult to reverse-engineer the ciphertext.

For instance, the Data Encryption Standard (DES) uses 16 rounds, while the Advanced Encryption Standard (AES) uses 10, 12, or 14 rounds depending on the key size.

Confusion and Diffusion

Confusion and diffusion are essential properties for a secure block cipher.

Confusion ensures that the relationship between the plaintext, ciphertext, and key is as complex as possible, making it difficult to deduce the key.

Diffusion ensures that a small change in the plaintext results in a significant change in the ciphertext, spreading the influence of each plaintext bit over many ciphertext bits.

Key Size

The key size determines the strength of the encryption.

Larger key sizes provide greater security against brute-force attacks. Common key sizes for block ciphers are 128, 192, and 256 bits.

For example, AES supports all three key sizes, providing flexibility and enhanced security.

Key Schedule

The key schedule is the algorithm that generates the subkeys used in each round of encryption.

A well-designed key schedule ensures that the subkeys are independent and unpredictable, preventing attacks that exploit weak keys or key-dependent properties of the cipher.

Block Size

The block size affects the security and efficiency of the cipher.

A larger block size reduces the risk of certain attacks, such as dictionary attacks, but may also increase the computational overhead.

A block size of 128 bits is generally considered secure and efficient for most applications.

Non-linearity and S-boxes

Non-linearity is crucial for creating confusion in the cipher. S-boxes (substitution boxes) are used to

introduce non-linearity by substituting input bits with output bits in a non-linear fashion.

This makes it harder for attackers to use linear approximations to break the cipher.

Avalanche Effect

A good block cipher should exhibit the avalanche effect, where a small change in the plaintext or key results in a significant change in the ciphertext.

This property ensures that the output is highly sensitive to changes in the input, enhancing the security of the cipher.

Security Analysis

A thorough security analysis is essential to evaluate the resistance of the cipher against various attacks, such as differential cryptanalysis, linear cryptanalysis, and brute-force attacks.

The cipher should also be tested for its resistance to implementation attacks, such as side-channel attacks.

In conclusion, the design of a block cipher involves balancing security, efficiency, and ease of implementation.

By adhering to these principles, cryptographers can create robust encryption algorithms that protect sensitive data from unauthorized access.

References:

1. Cryptography and Network Security (Textbook) William Stallings; Pearson Education
2. Cryptography & Network Security Behrouz A. Forouzan; Tata McGraw-Hill
3. Information Security Principles and Practice Deven Shah, Wiley-India
4. Information Security Principles and Practice Mark Stamp; Wiley India edition
5. Information systems security Nina Godbole; Wiley Publications, 200

3.3 Confusion & Diffusion

In cryptography, **confusion** and **diffusion** are two fundamental properties identified by Claude Shannon in his 1945 report "A Mathematical Theory of Cryptography". These properties are essential for creating secure ciphers and thwarting cryptanalysis methods.

Confusion

Confusion aims to obscure the relationship between the plaintext, ciphertext, and the encryption key.

This is typically achieved through substitution techniques, where each element of the plaintext is replaced with another element according to a complex set of rules.

The goal is to make the ciphertext appear as random as possible, making it difficult for an attacker to deduce the key even if they have some knowledge of the plaintext or ciphertext.

Example

A classic example of confusion is the **Caesar Cipher**, where each letter in the plaintext is shifted a certain number of places in the alphabet.

Plaintext: HELLO

Ciphertext: KHOOR (Shifted by 3)

Diffusion

Diffusion ensures that the influence of one plaintext digit is spread out over many ciphertext digits.

This is usually achieved through permutation or transposition techniques.

The aim is to mask patterns in the plaintext by spreading its statistical structure over the entire ciphertext, making it difficult for attackers to identify any patterns.

Example

In a block cipher like **DES**, diffusion ensures that a change in one bit of the plaintext affects many bits of the ciphertext.

Plaintext: 1010

Ciphertext: 1100 (After permutation)

Key Differences

- **Confusion** is achieved through substitution, while **diffusion** is achieved through transposition or permutation.
- **Confusion** obscures the relationship between the ciphertext and the key, whereas **diffusion** obscures the relationship between the plaintext and the ciphertext.
- Both stream and block ciphers use **confusion**, but only block ciphers use **diffusion**.

References:

1. Cryptography and Network Security (Textbook) William Stallings; Pearson Education
2. Cryptography & Network Security Behrouz A. Forouzan; Tata McGraw-Hill
3. Information Security Principles and Practice Deven Shah, Wiley-India
4. Information Security Principles and Practice Mark Stamp; Wiley India edition
5. Information systems security Nina Godbole; Wiley Publications, 200

3.4 Data Encryption Standard(DES)

Data Encryption Standard (DES) is a symmetric block cipher. By 'symmetric', we mean that the size of input text and output text (ciphertext) is same (64-bits).

The 'block' here means that it takes group of bits together as input instead of encrypting the text bit by bit.

Data encryption standard (DES) has been found vulnerable to very powerful attacks and therefore, it was replaced by Advanced Encryption Standard (AES).

- It is a block cipher that encrypts data in 64 bit blocks.
- It takes a 64-bit plaintext input and generates a corresponding 64-bit ciphertext output.
- The main key length is 64-bit which is transformed into 56-bits by skipping every 8th bit in the key.
- It encrypts the text in 16 rounds where each round uses 48-bit subkey.
- This 48-bit subkey is generated from the 56-bit effective key.
- The same algorithm and key are used for both encryption and decryption with minor changes.

Working of Data Encryption Standard (DES)

DES is based on the two attributes of Feistel cipher i.e. Substitution (also called confusion) and Transposition (also called diffusion). DES consists of 16 steps, each of which is called a round. Each round performs the steps of substitution and transposition along with other operations.



The encryption starts with a 64-bit plaintext that needs to be encrypted using a 64-bit key. Plaintext is passed to Initial Permutation function and key is permuted using Permuted Choice 1 (PC-1).

Initial Permutation

The 64-bit plaintext block is input into an Initial Permutation (IP) function that rearranges the order of bits. The order of bits is changed using predefined table. The IP table is a 8x8 matrix (64 entries) where

each entry specifies the new position of a bit from the original plaintext.

Initial Permutation Table							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Working of IP Table:

- The first bit of the permuted block is taken from the 58th bit of the original plaintext.
- The second bit comes from the 50th bit and so on.
- The last (64th) bit comes from the 7th bit of the original plaintext.

The initial permutation (IP) happens only once and it happens before the first round.

The permutation this function do is fixed and does not depend on the plaintext.

This rearranged 64-bit plaintext then go through 16 rounds. Each of this round uses a different 48-bit subkey from the previous round subkey. These subkeys are generated from 64-bit key.

Key Transformation

The 64-bit initial key is converted into 56-bit effective key. This 56-bit key further generates 48-bit subkeys for each of the 16 Feistel rounds.

Conversion of 64-bit Key into 56-bit Key

Initial key first go through Permuted Choice 1 (PC-1) which reduces the key to 56 bits. In PC-1 every eighth bit in key is discarded. That is bit positions 8, 16, 24, 32, 40, 48, 56, and 64 are discarded.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

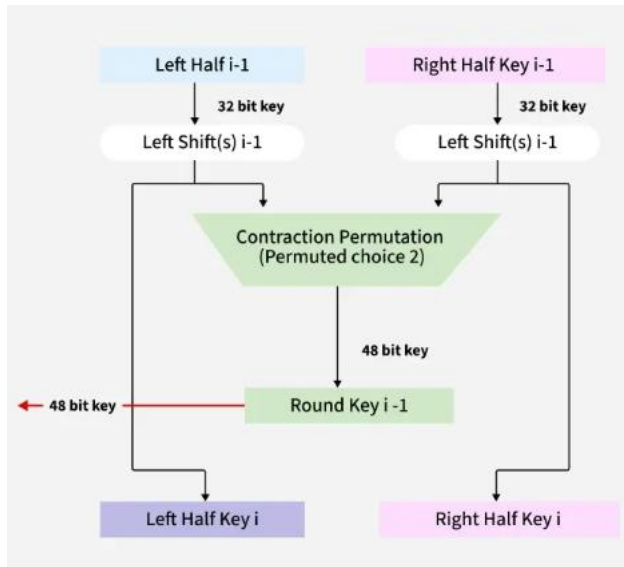
These discarded bits are called parity bits which are used for error checking. Remaining 56 bits are split into two 28-bit halves:

- Left Half (Ci): First 28 bits.
- Right Half (Di): Last 28 bits

Here i represent the number of the Feistel round.

Generating 48-bit Round Subkeys

For each of the 16 rounds, right half (Ci) and left half (Di) undergo circular left shift operation.



Key Transformation in DES

For Feistel round 1, 2, 9, and 16 both halves (left and right) undergo 1-bit left shift operation. For others rounds (3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15) the halves undergo 2-bit left shift operation.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
#key bits shifted	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Circular Left Shift Operation on Feistel Rounds

After circular shift operation is performed, C_i and D_i are again combined into 56-bit block. This block then go through Permutation Choice 2 (PC-2).

The PC-2 selects and arrange 48 bits out of the 56 to form the round subkey (K_i). These 48 bits are selected on the basis of a predefined table as shown below:

Permutation Choice 2 Table

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

According to this table 14th bit is placed to first position, 17th bit to second position, 11th bit to 3rd position and so on.

The output 48-bit subkey of this table is used to cipher the plaintext in the Feistel round. For next round we use already left shifted C_i and D_i as left and right half.

We again perform the circular left shift operation on both halves. We again combine the result into 56-bit block and use permutation choice 2 to contract this block into 48-bit subkey for next round.

The process of Circular Left Shift and Permutation Choice 2 is followed for 16 rounds and different round subkey (K_i) is generated for each Feistel Round.

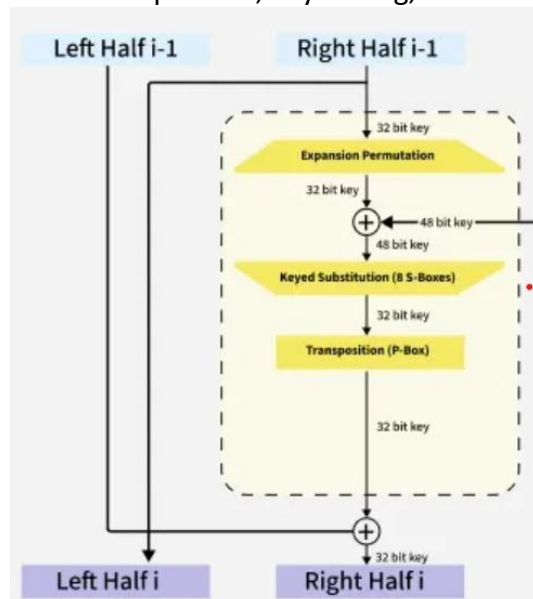
Each 48-bit subkey (K_i) is XORed with the expanded right half in the Feistel Round. Below is the explanation of what happens in every single Feistel round.

Feistel Rounds (1 – 16)

Every round receives 64-bits permuted plaintext from the Initial Permutation function and 48-bit transformed subkey (K_i).

The permuted 64-bit plaintext is divided into two halves called as Left Plaintext (LPT) and Right Plaintext (RPT). Both of these halves are 32 bit in size.

The right half or Right Plaintext (RPT) is processed using Mangler (F) function. Mangler (F) function involves expansion, key mixing, substitution (S-boxes), and permutation (P-box) of RPT.



The RPT first goes through Expansion Permutation. In this permutation 32-bit Right Plaintext (RPT) is expanded into 48 bits using expansion box or E-box table.

E-Box Expansion Table

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

The 48-bit expanded block is generated by arranging the bits as in E-Box table.

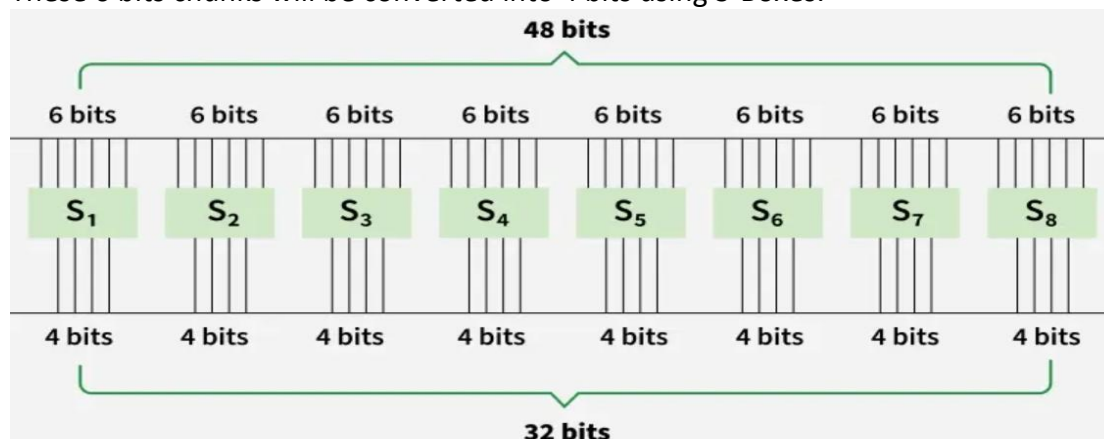
This expanded block is XORed ($\oplus$) with the 48-bit round subkey that we generated during key transformation process.

The XOR or Exclusive OR operation returns '0' as output if both inputs are same, else the out will be '1'. After XOR is performed, the resulting 48-bit block is split into eight chunks of 6-bit size each. Each of the chunk is then fed into a different S-box (S1 to S8).

For example, the output of XOR operation is converted into 6 bit chunks as follows:

101010 010001 011110 111010 100001 100110 010100 100111

These 6 bits chunks will be converted into 4 bits using S-Boxes.



S-Boxes are predefined lookup tables which reduces 6 bits chunk into 4 bits. Below is the list of these S-Boxes.

Suppose the first 6-bit chunk is 101010. We divide this chunk into two parts of 2 bit and 4 bit size. First and last bits are combined together for 2-bit part and rest bits make up the 4-bit part.

101010 -> (1)(0101)(0) -> divided into 10 and 0101

We look for these parts in the rows and columns of S1 table. The number in the cell where row is '10'

and column is '0101' is '6' in the S1 table. The binary value of six is '0110'. This is the 4 bit value that S-Box 1 generated from the 6 bit input '101010'.

6-bit chunk: '101010' converted into 4-bit chunk: '0110'

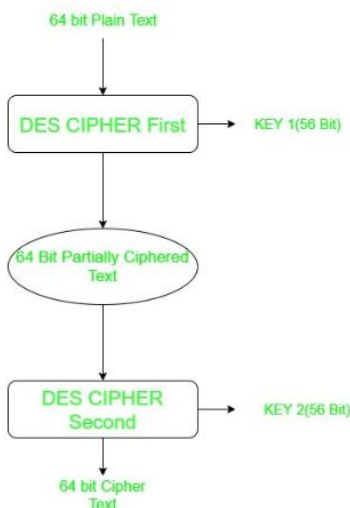
Similarly we convert every 6-bit chunk into 4-bit value using S-Boxes. This process is called substitution.

After that we combine all of these 4-bit chunks to get 32-bit block as output. This 32 bit again get permuted using following table.

Double DES:

Double DES is a encryption technique which uses two instance of DES on same plain text. In both instances it uses different keys to encrypt the plain text. Both keys are required at the time of decryption.

The 64 bit plain text goes into first DES instance which then converted into a 64 bit middle text using the first key and then it goes to second DES instance which gives 64 bit cipher text by using second key.

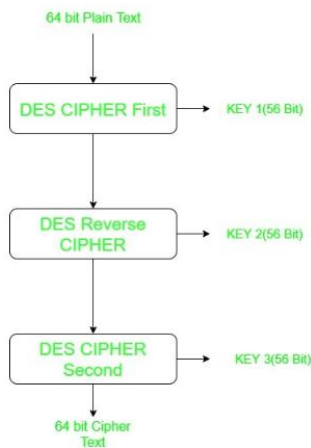


However double DES uses 112 bit key but gives security level of 2^{56} not 2^{112} and this is because of meet-in-the middle attack which can be used to break through double DES.

Triple DES:

Triple DES is a encryption technique which uses three instance of DES on same plain text.

It uses three different types of key choosing technique in first all used keys are different and in second two keys are same and one is different and in third all keys are same.



Triple DES is also vulnerable to meet-in-the middle attack because of which it give total security level of 2^{112} instead of using 168 bit of key.

The block collision attack can also be done because of short block size and using same key to encrypt large size of text. It is also vulnerable to sweet32 attack.

Strength of DES:

Key Length and Brute Force Attacks

One of the primary strengths of DES is its use of a 56-bit key. This key length allows for (2^{56}) possible keys, making a brute force attack theoretically impractical.

For instance, a machine performing one DES encryption per microsecond would take more than a thousand years to break the cipher.

However, this assumption is conservative. In 1998, the Electronic Frontier Foundation (EFF) demonstrated that DES could be broken in less than three days using a special-purpose machine.

Algorithmic Strength

DES is based on the Feistel network and involves 16 rounds of substitution and transposition. Each round uses a different 48-bit subkey derived from the original 56-bit key through a process of circular shifting and permutation.

This complexity makes it difficult to crack DES using cryptanalysis alone. However, the algorithm has been found vulnerable to certain statistical attacks like differential and linear cryptanalysis.

Timing Attacks

Another potential vulnerability of DES is timing attacks. These attacks exploit the time it takes for the system to respond to various inputs to extract secret data.

While timing attacks are more commonly associated with public key algorithms, they can also be applied to symmetric key algorithms like DES.

Alternatives to DES

Given the vulnerabilities of DES, it is advisable to consider more secure alternatives for modern data encryption needs.

Advanced Encryption Standard (AES) and Triple DES (3DES) are two such alternatives that offer stronger

security.

References:

1. Cryptography and Network Security (Textbook) William Stallings; Pearson Education
2. Cryptography & Network Security Behrouz A. Forouzan; Tata McGraw-Hill
3. Information Security Principles and Practice Deven Shah, Wiley-India
4. Information Security Principles and Practice Mark Stamp; Wiley India edition
5. Information systems security Nina Godbole; Wiley Publications, 200

3.5 Differential and Linear cryptanalysis

Cryptanalysis is the process of decoding encrypted communications without access to the actual key. It involves finding vulnerabilities in cryptographic systems to retrieve plaintext from ciphertext.

Two prominent techniques in cryptanalysis are **differential cryptanalysis** and **linear cryptanalysis**.

Differential Cryptanalysis

Differential cryptanalysis is a method used to analyze and break encryption systems by examining the differences between pairs of plaintexts and their corresponding ciphertexts.

This technique was first defined by Eli Biham and Adi Shamir in 1990.

How It Works

Differential cryptanalysis studies how differences in the input (plaintext) affect the differences in the output (ciphertext). The process involves:

1. Selecting pairs of plaintexts that differ by a specific bit.
2. Analyzing the differences in the resulting ciphertexts.
3. Establishing a high correlation between the differences in plaintext and ciphertext to deduce the encryption key.

Advantages

- Effective against ciphers using non-linear substitution boxes (S-boxes).
- Requires fewer plaintext-ciphertext pairs compared to other techniques.

Limitations

- Less effective against ciphers with small block sizes or limited rounds.
- Requires many plaintext-ciphertext pairs with specific characteristics.

Example

Consider a two-round AES cipher. By analyzing the differences in plaintext and ciphertext blocks, we can derive differential characteristics to find the encryption key.

Linear Cryptanalysis

Linear cryptanalysis is a technique that finds linear approximations to predict input-output bit relationships in cryptographic functions. It was invented by Matsui and Yamagishi in 1992.

How It Works

Linear cryptanalysis constructs linear equations using known plaintext and ciphertext combinations to obtain the encryption key.

It examines the linear connection between the plaintext, ciphertext, and encryption key.

Advantages

- Simple to implement and requires minimal computational resources.
- Can target a wide variety of cryptographic systems.

Limitations

- Less effective against ciphers using non-linear S-boxes.
- Requires a considerable amount of known plaintext-ciphertext pairs.

Example

An attack on a small, two-round DES encryption can be performed using linear approximations of plaintext and ciphertext pairs to derive linear equations and solve for the encryption key.

Comparison

Strengths and Weaknesses

- **Linear Cryptanalysis:** Easy to implement, requires large data, less effective against non-linear S-boxes.
- **Differential Cryptanalysis:** Effective against non-linear S-boxes, requires fewer data, more complex to implement.

Application and Suitability

- **Linear Cryptanalysis:** More effective on symmetric ciphers with many rounds.
- **Differential Cryptanalysis:** Better suited for block ciphers with fewer rounds.

Effectiveness

Both techniques are effective in breaking encryption systems, but their success depends on the specific system and the quality and amount of data available for analysis.

In conclusion, both differential and linear cryptanalysis are valuable tools in cryptanalysis, each with its strengths and weaknesses. Combining these techniques can provide a comprehensive assessment of a cryptographic system's strength

References:

1. Cryptography and Network Security (Textbook) William Stallings; Pearson Education
2. Cryptography & Network Security Behrouz A. Forouzan; Tata McGraw-Hill
3. Information Security Principles and Practice Deven Shah, Wiley-India
4. Information Security Principles and Practice Mark Stamp; Wiley India edition
5. Information systems security Nina Godbole; Wiley Publications, 200

